SAVEETHA SCHOOL OF ENGINEERING

OPERATING SYSTEMS

ASSIGNMENT | CO1

EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

Student Details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Department	B.E. Computer Science and Engineering (Cyber Security)
Course	CSA0421 – Operating Systems
Assignment	Assignment 1 (Tasks 7)

TASK 7 – EVALUATE PROCESS-CONTROL SYSTEM CALLS

7.1 fork(), exec(), wait() and exit()

Call	Function	Use in a shell
fork()	Creates a child process. The child initially receives a copy-on-write view of the parent address space.	Creates a separate process in which the command can run.
exec() family	Replaces the current process image with a new program; the PID remains the same.	The child becomes the requested command, such as expr.
wait() / waitpid()	Waits for a child to change state and collects its termination status.	The parent shell waits for a foreground command and obtains its exit code.
exit() / _exit()	Terminates a process. exit() performs C library cleanup and flushes stdio streams; _exit() terminates immediately at the process level without flushing stdio buffers.	A command returns its status; a child that failed to exec can use _exit(127).

7.2 How a shell executes a command

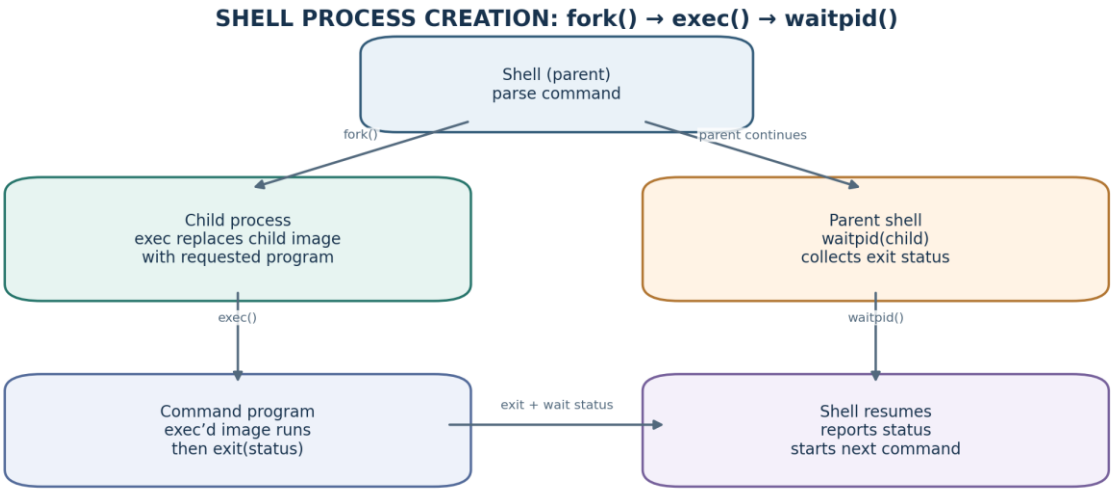


Figure 5. The child becomes the command through exec(); the parent shell waits and collects its exit status.

1. The shell reads and parses a command line, identifies the executable and its arguments, and prepares any requested redirections or pipelines.
2. For a foreground command, the shell calls fork(). The kernel creates a child process with a separate process identity and copy-on-write address-space mappings.
3. The child calls an exec() function such as execlp(). If successful, the child's old program image is replaced by the command program. If exec fails, the child should report the error and terminate with a failure status such as 127.
4. The parent calls waitpid() for the child. The kernel reports whether it exited normally or was terminated by a signal.
5. The shell extracts WEXITSTATUS(status) only when WIFEXITED(status) is true, stores the result as the command status and displays a prompt or starts the next command.

7.3 Zombie and orphan processes

A zombie is a child that has terminated but whose parent has not yet collected its termination status. The process has finished running, but a small process-table entry remains until wait() or waitpid() collects it. A shell should wait for its foreground children and arrange to reap background children, often using SIGCHLD handling or a wait loop.

An orphan is a child whose parent exits while the child is still running. The operating system reparents it to a designated system process or subreaper, which can later collect its exit status. An orphan is not necessarily a malfunction; it is a process-lifecycle condition. Long-running orphaned jobs should still be managed deliberately.

7.4 Why fork() followed by exec()?

The two-step design separates process creation from program selection. After fork(), the child can adjust file descriptors, set up pipes, apply redirections, change environment variables or configure process attributes before exec(). This flexibility is important for a shell. Modern systems also offer posix_spawn() as a more direct interface for common process-launch patterns.

7.5 Completed C program – TODO 1

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

static int wait_for(pid_t pid, int *status) {
    while (waitpid(pid, status, 0) < 0) {
        if (errno == EINTR) continue;
        return -1;
    }
    return 0;
}

int main(void) {
    pid_t pid;
    int status;

    printf("shell: running job 1\n");
    fflush(stdout);

    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        perror("execlp");
        _exit(127);
    }
    if (wait_for(pid, &status) < 0) { perror("waitpid"); return 1; }
    if (WIFEXITED(status))
        printf("job 1 exit status %d\n", WEXITSTATUS(status));
    else
```

```
        printf("job 1 did not exit normally\n");

    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) _exit(3);

    if (wait_for(pid, &status) < 0) { perror("waitpid"); return 1; }
    if (WIFEXITED(status))
        printf("job 2 exit status %d\n", WEXITSTATUS(status));
    else
        printf("job 2 did not exit normally\n");
    return 0;
}
```

7.6 Compile and expected output

```
gcc -std=c11 -Wall -Wextra -O2 task07_shell.c -o task07_shell
./task07_shell

shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

7.7 Why stdout is flushed before fork()

When stdout is connected to a terminal, it is commonly line-buffered, but when redirected to a file it is commonly fully buffered. If a line is still in the C library buffer when fork() occurs, the child receives a copy of that buffer. If both parent and child later call exit(), both copies may be flushed, so the text can appear twice in the output file. fflush(stdout) before fork() empties the parent's buffer before it is duplicated.

The assignment's experiment asks to remove the second fflush(stdout), replace _exit(3) with exit(3), and redirect output to a file. The text "shell: running job 2" then remains buffered at the time of fork(). The child calls exit(3) and flushes its inherited copy; the parent later

flushes its own copy. The line therefore appears twice. _exit(3) avoids the C library flush in the child.

7.8 Alternative: posix_spawn() or threads

posix_spawn() combines process creation and program execution in one API and can be efficient for a shell that needs standard file actions and attributes. It may avoid some of the overhead of a general fork() path, depending on the platform and implementation. It still creates a separate process and preserves command isolation. Threads share one process address space, so launching commands directly as threads would not provide the same memory and privilege isolation; an external executable still needs a process image. For 1,000 short commands, process creation, scheduling and program loading dominate more than the source-level API name. Benchmark the exact workload rather than asserting a universal time advantage.

fork() uses copy-on-write, so it does not normally copy every parent memory page immediately. Page tables and kernel task structures still need resources, and pages are copied if modified. Threads generally share code, heap and global data, so thread creation is usually lighter, but shared memory introduces synchronization and safety concerns.